

DBMS Indexing: Ordered Indices and Their Variants

Introduction

To enable efficient data retrieval, database systems rely on **index structures**. An *ordered index* stores search-key values in sorted order, much like a library catalog. Each search key is associated with pointers to the corresponding records, supporting both **sequential processing** and **fast random access**.

Key Idea

Ordered indices minimize the number of block accesses required to locate a record, significantly improving query performance.

Clustering and Nonclustering Indices

- **Clustering Index (Primary Index):** File is physically ordered on the search key (often the primary key).
- **Nonclustering Index (Secondary Index):** File order differs from the index search key.

Files ordered sequentially on the search key, with a clustering index, are called **index-sequential files**.

Dense and Sparse Indices

An index entry = *search-key value* + *pointer to record(s)*. Two types:

Dense Index

- Entry for every search-key value.
- Clustering: index points to the first record of each key.
- Nonclustering: index must maintain a list of pointers.

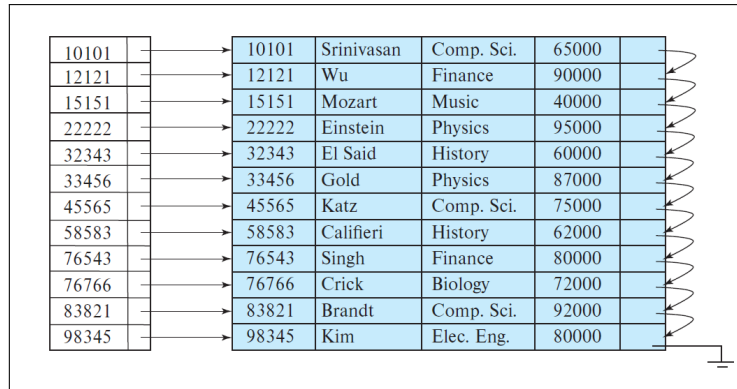


Figure 1: *Dense Index Example*

Sparse Index

- Contains entries for only *some* search-key values.
- Works only with clustering indices.
- Each entry points to the first record of its block.

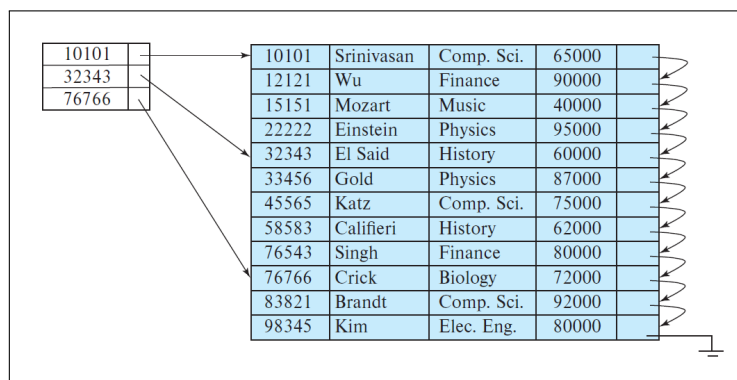


Figure 2: *Sparse Index Example*

Trade-off

Dense Index: Faster access, but higher space and maintenance cost.

Sparse Index: Slower lookups, but reduced space and updates.

Multilevel Indices

Large files may produce large indices. To optimize:

- Treat the index as a sequential file.
- Build a sparse index on top (recursively if needed).

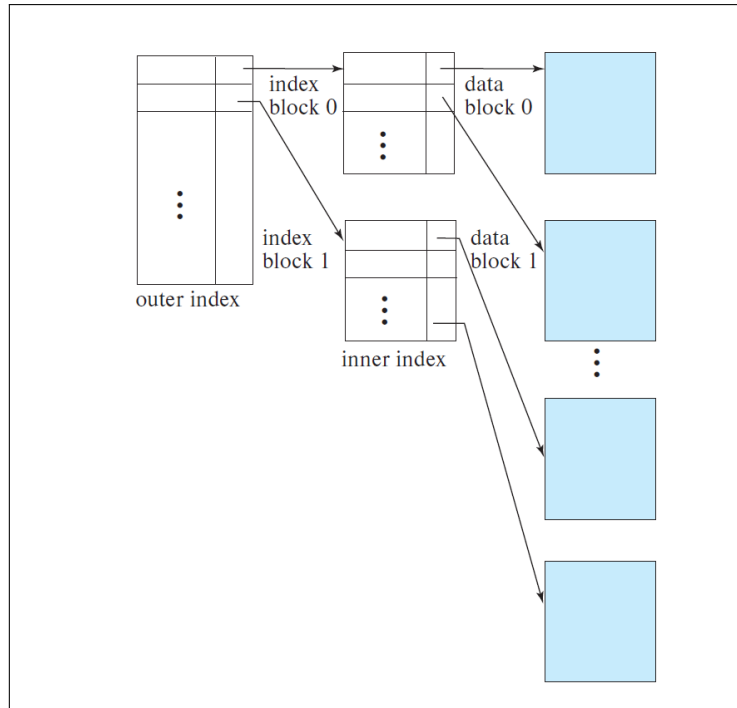


Figure 3: *Two-Level Sparse Index*

Advantage

Each additional index level reduces I/O overhead and improves search performance.

Example: Multilevel Index Efficiency

Suppose we build a dense index on a relation with **1,000,000 tuples**.

- 100 index entries fit in a 4KB block.
- Index size = 10,000 blocks.
- Binary search requires $\lceil \log_2(10000) \rceil = 14$ random reads.
- On a disk with 10ms/block access, search = 140ms.

With an outer sparse index:

- Outer index = 100 blocks.
- If kept in memory, only **1 block read** is needed.
- Search is $\approx 14\times$ faster.

Key Insight

As data grows (e.g., 100,000,000 tuples), even outer indices may be large. This motivates **multi-level indices**, which scale efficiently with file size.

Index Updates

Indices must be updated whenever a record is **inserted**, **deleted**, or an indexed attribute is **modified**. Record updates are equivalent to a **deletion of the old record** followed by an **insertion of the new record**. Thus, we explicitly handle only *insertion* and *deletion*.

Insertion

Insertion Rules

Dense Index:

- If the search-key does not exist → insert a new index entry.
- If it exists:
 - If storing pointers to all records → add pointer to new record.
 - If storing only first pointer → place new record after others.

Sparse Index:

- If a new block is created → insert its first search-key in index.
- If the new record is the smallest in its block → update index entry.
- Otherwise → no update required.

Deletion

Deletion Rules

Dense Index:

- If deleted record was the *only one* with that key → remove index entry.
- Otherwise:
 - If all pointers stored → remove pointer to deleted record.
 - If only first pointer stored & deleted record was first → update pointer to next record.

Sparse Index:

- If no entry exists for deleted key → no change.
- Otherwise:
 - If it was the only record → replace entry with next search-key (or remove if already present).
 - If entry pointed to deleted record → update pointer to next record with same key.

Note on Multilevel Indices

In **multilevel indices**, these rules apply recursively:

- Lowest-level index is updated first.
- Higher levels are updated only if changes propagate upward.

Secondary Indices

- Always **dense**.
- Support queries on non-clustering attributes.
- May use **buckets** to handle duplicate keys.

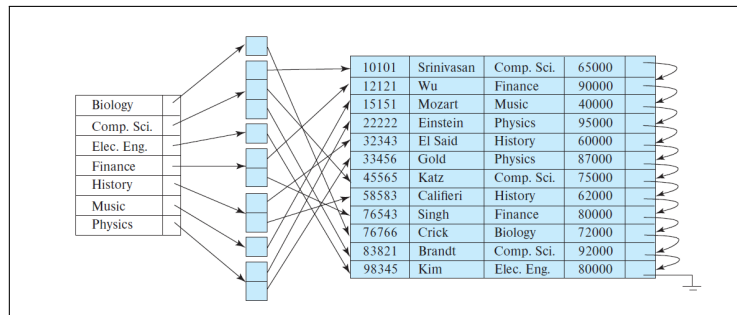


Figure 4: *Secondary Index with Buckets*

Indices on Multiple Keys

Composite keys involve multiple attributes, e.g., (`course_id`, `semester`, `year`). Ordered **lexicographically**:

$$(a_1, a_2) < (b_1, b_2) \iff (a_1 < b_1) \vee (a_1 = b_1 \wedge a_2 < b_2).$$

Use Case

Composite indices enable efficient multi-attribute queries (e.g., finding students enrolled in a given course in a specific semester).

Conclusion

Ordered indices balance between **dense vs sparse**, **clustering vs secondary**, and **multilevel structures**. Choosing the right mix enables efficient queries while minimizing storage and update overhead.